

智航三机自主搜索系统

V6.7.19 Portable

跨电脑部署、YOLO 真实运行、竞赛任务执行与交付说明书

模板基线：V6.7.18

目标平台：Ubuntu 20.04 / ROS Noetic / Gazebo 11 / PX4 / XTDrone

任务对象：3 架 standard_vtol，6 个静态目标，2 个动态目标

日期：2026 年 7 月 23 日

正式模式：视觉定位与飞行器状态闭环；Gazebo 目标真值不进入规划和控制

文档信息

版本	V6.7.19 Portable
代码基线	V6.7.18 任务算法
交付内容	完整代码包、Markdown、DOCX、PDF、校验文件
正式启动原则	三路真实处理链持续 $\geq 10\text{Hz}$ 后，先启动 bag，再启动目标运动，最后授权解锁
受保护官方文件	PX4/Gazebo 赛事资源、XTDrone 通信、model_state.py、zhihang_ws
验证边界	容器完成静态与单元测试；目标电脑需完成 GPU、ROS、飞行和官方消息联调

目录

跨电脑部署、YOLO 真实运行、竞赛任务执行与交付说明书

1. 交付目标与设计原则

2. 赛事要求基线

- 2.1 环境与自主性
- 2.2 目标集合
- 2.3 官方结果话题
- 2.4 计时和定位评分约束
- 2.5 score1.bag

3. 版本结构与模块职责

- 3.1 官方环境层
- 3.2 Portable 适配层
- 3.3 自主任务应用层

4. 支持的电脑配置

- 4.1 推荐正式比赛配置
- 4.2 双 GPU 或三 GPU
- 4.3 低显存 GPU
- 4.4 CPU 电脑
- 4.5 无桌面/远程电脑
- 4.6 外部 YOLO 计算机

5. 机器 profile 详解

6. 安装前备份

- 6.1 备份旧版本交付包
- 6.2 备份当前 ROS 包和编译产物
- 6.3 记录受保护通信文件哈希

7. YOLO 环境配置

- 7.1 复用已有 conda 环境
- 7.2 新建 conda 环境
- 7.3 venv 环境
- 7.4 system 或 direct Python
- 7.5 模型类别检查
- 7.6 三进程并发容量测试
- 7.7 导出可复现环境

8. 完整安装流程

- 8.1 解压和包内自检
- 8.2 生成 profile
- 8.3 安装与编译
- 8.4 全面诊断

9. 正式任务全流程

- 9.1 启动命令
- 9.2 自动启动顺序
- 9.3 初始角色和航线
- 9.4 动态目标发现后的角色重分配
- 9.5 动态跟踪控制
- 9.6 动态目标丢失和重捕获
- 9.7 suv_camo/prius_hybrid_camo 混淆保护
- 9.8 静态搜索和候选管理
- 9.9 静态精准定位
- 9.10 返航与结束

10. 验证模式与正式模式

- 正式模式
- 验证模式

11. 运行状态与关键日志

- 11.1 正常启动标志
- 11.2 动态跟踪关键事件
- 11.3 静态精准定位关键事件
- 11.4 官方输出标志

12. 输出目录与提交材料

13. 常见故障处理

- 13.1 模型路径显示为`model=~/...`
- 13.2 `rospack: package 'zhihang\_adaptive\_search\_v6' not found`
- 13.3 找不到`adaptive\_search\_formal.yaml`
- 13.4 找不到官方消息类型
- 13.5 CUDA 不可用
- 13.6 CUDA OOM
- 13.7 三路处理链不足 10Hz
- 13.8 QGC 显示`Invalid offboard setpoint`
- 13.9 动态跟踪反复调 yaw 后丢失
- 13.10 静态精准定位陷入固定翼 HOLD

14. 赛前完整检查表

- T-1 天
- 比赛启动前
- 任务结束后

15. 回滚

16. 验证状态与限制

17. 完整代码索引

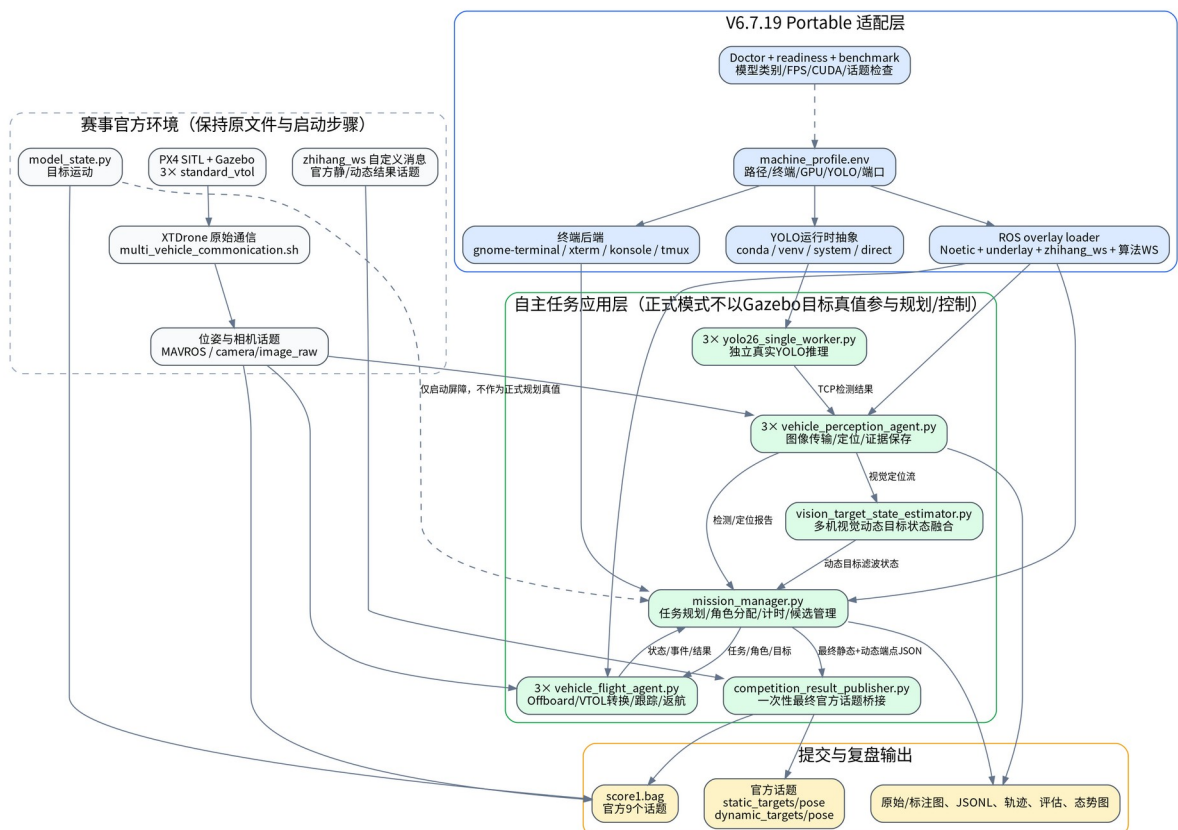
1. 交付目标与设计原则

V6.7.19 不是重新设计 V6.7.18 的搜索与控制算法，而是在其已验证逻辑外增加一层可移植部署框架，使同一套代码能够迁移到不同用户名、目录结构、ROS 工作空间、显卡数量、显存大小、YOLO 环境和终端程序的 Ubuntu 电脑。

本版本遵循六项原则：

- 赛事官方环境优先。** PX4 模型、Gazebo 模型、world、赛事 launch、XTDrone 通信、位姿脚本、`model_state.py` 和 `zhihang_ws` 继续使用赛方原件。
- 修改工程前先备份。** 安装脚本先备份现有算法 ROS 包，再编译覆盖；用户手工操作也必须先执行本手册的备份命令。
- 正式模式视觉闭环。** Gazebo 目标真值不进入目标定位、规划、任务分配或飞行控制；其仅按官方要求记录进 `score1.bag`。
- 三个真实 YOLO 链路独立。** 每架无人机拥有独立感知代理、YOLO 进程、端口、性能状态和日志，任一链路未达到持续 10Hz，正式任务不越过启动屏障。
- 第一次官方消息就是最终结果。** 评分系统以官方话题第一次接收消息为准，因此本系统在全部任务结束后只发布一次整理后的静态和动态目标结果。
- 安全落地优先。** 搜索到 32 分钟立即进入返航，预留 3 分钟；即使总时间超过 35 分钟，程序只告警而不中止返航和降落监督。

V6.7.19 跨电脑部署与三机任务架构



V6.7.19 总体架构

2. 赛事要求基线

本节依据用户提供的《2026 年度中国青年科技创新“揭榜挂帅”擂台赛——XH-202632 无人机集群自主协同识别与态势融合参赛选手手册》归纳。若后续赛方发布修订版，以最新官方文件为准。

2.1 环境与自主性

- 操作系统为 Ubuntu 20.04，仿真平台为 Gazebo，机器人系统为 ROS1。
- 使用 3 架无人机，在复杂野外环境中自主完成搜索、识别、定位、态势生成和指定区域降落。
- 全流程无需人工参与。
- 官方文件必须按规定位置部署，比赛期间不得随意关闭或重启官方进程。

2.2 目标集合

静态目标模型共 6 类：

- prius_hybrid
- car_lexus
- car_opel
- person_white
- prius_hybrid_camo
- fire_truck

科目一发布时，其类别均为 `static_target`。动态目标共 2 类：

- suv_camo
- person_red

科目二态势图和动态结果需保留动态目标模型名。

2.3 官方结果话题

静态话题：

```
/zhihang2026/static_targets/pose
```

动态话题：

```
/zhihang2026/dynamic_targets/pose
```

外层消息包含 `header_timestamp` 和 `categories`；每条结果包含：

```
timestamp
model_name
x
y
```

每个话题最多 20 条结果，评分以系统第一次接收到的消息为准。V6.7.19 在任务完成后从最终结果生成一次性消息：静态目标优先采用悬停精确定位，未完成精确定位时采用当前最高置信度的可用检测定位；动态目标优先采用完整轨迹的首末端点，缺少本机轨迹时可使用可靠的跨机注入或最近视觉定位

作为末端保底。静态结果最多 6 条，动态起止端点最多 4 条。若一个结果分量为空而另一个有效，仍发布两类官方消息，以保住有效分量得分；只有静态和动态结果同时为空时才拒绝发布。

2.4 计时和定位评分约束

- 计时从第一架飞机在起飞区域内 `armed` 开始，到最后一架在降落区域内 `disarmed` 结束。
- 总门限为 35 分钟。
- 静态定位距离在水平面计算，定位误差达到 10m 时该目标定位项为 0。
- 系统内部精确定位控制目标设置为 XY 误差不超过 5m，以给定位评分留出余量。

2.5 score1.bag

官方要求在无人机起飞前开始记录，并包含以下 9 个话题：

```
/standard_vtol_0/mavros/state
/standard_vtol_1/mavros/state
/standard_vtol_2/mavros/state
/gazebo/model_states
/xtldrone/standard_vtol_0/cmd
/xtldrone/standard_vtol_1/cmd
/xtldrone/standard_vtol_2/cmd
/zhihang2026/static_targets/pose
/zhihang2026/dynamic_targets/pose
```

V6.7.19 的正式一键脚本在三路 YOLO 准备完成后，先启动 bag，再启动目标运动，再授权管理器解锁，因此仍满足“起飞前记录”。

赛事要求与代码位置的逐项映射见 [COMPETITION_COMPLIANCE_MATRIX.md](#)。

3. 版本结构与模块职责

3.1 官方环境层

本层由赛方或既有环境提供，本包不覆盖：

- PX4 SITL 和赛事 Gazebo 场景；
- [multi_vehicle_commonication.sh](#) 或环境中同义通信脚本；
- 位姿真值转发脚本；
- 三路相机话题；
- [model_state.py](#)；
- [zhihang_ws](#) 自定义消息；
- QGroundControl。

3.2 Portable 适配层

模块	作用
machine_profile.env	保存本机所有路径、终端、YOLO、GPU 和端口参数

模块	作用
<code>configure_portable_machine.py</code>	自动检测并生成 profile 和检测报告
<code>load_machine_profile.sh</code>	为所有脚本统一加载 profile
<code>source_zhihang_ros_env.sh</code>	按正确顺序加载 Noetic、underlay、 <code>zhihang_ws</code> 和算法工作空间
<code>terminal_launcher_common.sh</code>	选择图形终端或 tmux 并保持终端日志可见
<code>yolo_runtime_common.sh</code>	封装 conda、venv、system 和 direct Python
<code>setup_yolo_runtime.sh</code>	创建/安装/验证 YOLO 运行环境
<code>doctor_portable.sh</code>	检查路径、ROS、官方消息工作空间、模型、CUDA、端口和配置
<code>benchmark_three_yolo_capacity.sh</code>	并发加载 3 个模型实例，评估本机三路推理容量
<code>export_yolo_runtime_manifest.sh</code>	导出软件版本、GPU 信息、pip freeze、模型 SHA-256

3.3 自主任务应用层

`mission_manager.py`

负责：

- 生成三机差异化航线；
- 任务启动屏障；
- 动态/机动/静态角色管理；
- 动态目标任务分配、临时锁定和回滚；
- 静态候选聚类、冲突隔离、精准验证排序；
- 32+3 分钟计时；
- 统一返航指令；
- 最终结果、评估和态势数据汇总；
- 生成一次性官方结果 JSON。

`vehicle_flight_agent.py`

每架飞机各运行一个，负责：

- Offboard 预发送、解锁、起飞和 VTOL 转换；
- 固定翼航段直线巡航；

- 动态目标截获、旋翼跟踪和重捕获；
- 静态目标固定翼接近、旋翼精准悬停；
- HOLD/OFFBOARD 异常恢复；
- 固定翼无效零速度 setpoint 保护；
- 固定翼优先独立返航、转旋翼和自动降落。

vehicle_perception_agent.py

每架飞机各运行一个，负责：

- 接收相机图像并发送给本机 YOLO 进程；
- 处理检测框；
- 相机投影和目标地面定位；
- 多帧有效性、置信度和空间一致性判断；
- 保存原始图像、标注图和误检/候选证据；
- 输出感知 FPS、检测报告和定位报告。

yolo26_single_worker.py

每架飞机一个独立进程：

- 通过 TCP 接收 JPEG 图像；
- 运行真实 YOLO 权重；
- 返回类别、置信度和检测框；
- 支持 [auto/cpu/GPU 编号](#) 设备；
- 支持 640、512、416 自适应输入尺寸；
- 运行时不支持 [quantize](#) 参数时自动退回标准推理；
- 独立记录 FPS 和日志。

vision_target_state_estimator.py

正式模式下融合多机视觉定位报告，形成动态目标状态流。它不订阅 Gazebo 目标真值。

competition_result_publisher.py

- 从管理器接收最终静态和动态结果 JSON；
- 从已 source 的 [zhihang_ws](#) 中发现具备指定字段结构的自定义消息类型；
- 也允许在 YAML 中显式指定消息类型；
- 只发布第一次、也是最终一次官方静态/动态话题；
- 记录 [competition_topic_publish_evidence.json](#) 作为提交证据。

4. 支持的电脑配置

4.1 推荐正式比赛配置

- Ubuntu 20.04 桌面；
- NVIDIA GPU，CUDA 版 PyTorch；
- 16GB 以上内存；
- SSD；
- 单 GPU 显存建议 8GB 或以上；
- 三路相机约 20Hz，三路处理链持续 10Hz 以上。

单 GPU profile:

```
export ZHIHANG_YOLO_DEVICES="0,0,0"
export ZHIHANG_YOLO_QUANTIZE="16"
export ZHIHANG_YOLO_ADAPTIVE="1"
export ZHIHANG_YOLO_ADAPTIVE_SIZES="640,512,416"
```

4.2 双 GPU 或三 GPU

双 GPU 示例:

```
export ZHIHANG_YOLO_DEVICES="0,1,1"
```

三 GPU 示例:

```
export ZHIHANG_YOLO_DEVICES="0,1,2"
```

每个 YOLO 进程独立加载模型，因此多 GPU 可以降低互相抢占和显存峰值。

4.3 低显存 GPU

优先按以下顺序处理:

7. 保持三个独立进程，启用自适应尺寸；
8. 将初始尺寸改为 512:

```
export ZHIHANG_YOLO_ADAPTIVE_SIZES="512,416"
```

9. 模型允许时使用更小模型；
10. 降低其他图形程序占用，QGC 可在完成检查后最小化；
11. 使用并发容量脚本验证，不得只看单模型 FPS。

4.4 CPU 电脑

CPU 模式主要用于代码验证和非正式调试:

```
export ZHIHANG_YOLO_REQUIRE_CUDA="0"
export ZHIHANG_YOLO_DEVICES="cpu,cpu,cpu"
export ZHIHANG_YOLO_QUANTIZE="32"
```

只有在三路真实相机处理链已经实测持续达到 10Hz 时，才应考虑 CPU 正式运行。合成黑图 FPS 不能替代真实相机、JPEG、TCP、YOLO、定位和 ROS 发布的端到端 FPS。

4.5 无桌面/远程电脑

```
export ZHIHANG_TERMINAL_BACKEND="tmux"
export ZHIHANG_HEADLESS="1"
export ZHIHANG_START_QGC="0"
```

查看终端：

```
tmux attach -t zhihang_v6_7_19
```

Gazebo 和 QGC 本身仍可能需要 X11 或远程桌面，因此“无头”主要用于算法端或外部 YOLO 计算节点。

4.6 外部 YOLO 计算机

当三路 YOLO 运行在另一台局域网电脑时：

```
export ZHIHANG_YOLO_START_LOCAL_WORKERS="0"
export ZHIHANG_YOLO_HOSTS="192.168.1.50,192.168.1.50,192.168.1.50"
export ZHIHANG_YOLO_PORT_BASE="17771"
```

外部计算机分别运行：

```
bash run_detached_yolo_worker.sh 0 /path/to/best.pt 17771
bash run_detached_yolo_worker.sh 1 /path/to/best.pt 17772
bash run_detached_yolo_worker.sh 2 /path/to/best.pt 17773
```

正式比赛前必须验证网络时延、丢包、端口防火墙和三路端到端 FPS。远程节点时间戳以发送图像头部和 ROS 端记录为准，不应依赖两机系统时间直接相减。

5. 机器 profile 详解

复制模板：

```
cd ~/下载/zhihang_adaptive_search_v6_7_19_portable_bundle
cp machine_profile.example.env machine_profile.env
```

更推荐自动生成：

```
bash configure_portable_machine.sh \
  --workspace "$HOME/xt drone_competition_ws" \
  --xt drone-root "$HOME/XTDrone" \
  --px4-root "$HOME/PX4_Firmware" \
  --model "$HOME/yolo_models/best.pt" \
  --runtime auto \
  --record-bag
```

关键变量：

变量	说明
ZHIHANG_WS	算法 catkin 工作空间
ZHIHANG_ROS_SETUP	Noetic setup
ZHIHANG_OPTIONAL_UNDERLAY	PX4/Gazebo 或项目 underlay，可为空
ZHIHANG_MESSAGE_WS_SETUP	官方 <code>zhihang_ws/devel/setup.bash</code>

变量	说明
ZHIHANG_XTDRONE_ROOT	XTDrone 根目录
ZHIHANG_PX4_ROOT	PX4 根目录，仅用于诊断和记录
ZHIHANG_PX4_ROS_PACKAGE	一般为 <code>px4</code>
ZHIHANG_PX4_LAUNCH_FILE	<code>auto</code> 、 <code>zhihang2026.launch</code> 或实际文件名
ZHIHANG_POSE_SCRIPT	官方位姿启动脚本
ZHIHANG_MODEL_STATE_SCRIPT	官方目标运动代码
ZHIHANG_COMPETITION_DATA_DIR	<code>score1.bag</code> 保存目录
ZHIHANG_TERMINAL_BACKEND	<code>auto/gnome-terminal/xterm/konsole/tmux</code>
ZHIHANG_YOLO_RUNTIME	<code>auto/conda/venv/system/direct</code>
ZHIHANG_YOLO_MODEL	权重绝对路径或 <code>\$HOME</code> 路径
ZHIHANG_YOLO_DEVICES	三机对应设备 CSV
ZHIHANG_YOLO_REQUIRE_CUDA	正式默认 1
ZHIHANG_YOLO_START_LOCAL_WORKERS	本地起 YOLO 为 1，外部节点为 0
ZHIHANG_YOLO_HOSTS	三机 YOLO 主机 CSV
ZHIHANG_YOLO_PORT_BASE	默认 17771，后两机自动+1/+2

路径可包含空格，但 profile 内必须使用引号。V6.7.19 会清理参数前后空格并安全展开`~/`、`$HOME/`和`${HOME}/`，不会使用 `eval`。

6. 安装前备份

6.1 备份旧版本交付包

```

STAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p ~/xtdrone_backups

cp -a ~/下载/zhihang_adaptive_search_v6_7_18_bundle \
~/xtdrone_backups/zhihang_adaptive_search_v6_7_18_bundle_${STAMP}

```

6.2 备份当前 ROS 包和编译产物

```

STAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p ~/xtdrone_backups

```

```
if [ -d ~/xtdrone_competition_ws/src/zhilang_adaptive_search_v6 ]; then
  cp -a ~/xtdrone_competition_ws/src/zhilang_adaptive_search_v6 \
    ~/xtdrone_backups/zhilang_adaptive_search_v6_before_v6_7_19_${STAMP}
fi

tar -czf ~/xtdrone_backups/xtdrone_competition_ws_build_${STAMP}.tar.gz \
  -C ~/xtdrone_competition_ws build devel 2>/dev/null || true
```

6.3 记录受保护通信文件哈希

```
sha256sum \
~/XTDrone/communication/vtol_communication.py \
~/XTDrone/communication/multi_vehicle_communication.sh \
~/XTDrone/communication/multi_vehicle_communication.sh \
2>/dev/null | tee ~/xtdrone_backups/official_communication_${STAMP}.sha256
```

自动安装器会再次备份和校验，但手工备份仍建议保留。

7. YOLO 环境配置

7.1 复用已有 conda 环境

```
conda env list
conda run -n yolo26 python -c \
  "import torch,cv2,ultralytics; print(torch.__version__, torch.cuda.is_available(), ultralytics.__version__)"
profile:
```

```
export ZHIHANG_YOLO_RUNTIME="conda"
export ZHIHANG_YOLO_ENV="yolo26"
```

验证真实模型：

```
bash setup_yolo_runtime.sh \
  --runtime conda \
  --env yolo26 \
  --model "$HOME/yolo_models/best.pt"
```

7.2 新建 conda 环境

```
bash setup_yolo_runtime.sh \
  --create --install \
  --runtime conda \
  --env yolo26 \
  --python-version 3.10 \
  --model "$HOME/yolo_models/best.pt"
```

PyTorch CUDA 轮应与本机驱动兼容。需要指定下载源时：

```
bash setup_yolo_runtime.sh \
  --create --install \
  --runtime conda \
  --env yolo26 \
  --model "$HOME/yolo_models/best.pt" \
  --torch-index-url <与本机 CUDA 兼容的 PyTorch 官方 wheel 索引>
```

本包不会猜测具体 CUDA wheel 地址，因为不同电脑驱动和 PyTorch 版本不同。应先通过 [nvidia-smi](#) 确认驱动，再选择对应 PyTorch 发行版。

7.3 venv 环境

```
sudo apt install python3-venv

bash setup_yolo_runtime.sh \
  --create --install \
  --runtime venv \
  --venv "$HOME/.venvs/yolo26" \
  --model "$HOME/yolo_models/best.pt"
```

7.4 system 或 direct Python

系统 Python：

```
export ZHIHANG_YOLO_RUNTIME="system"
```

指定解释器：

```
export ZHIHANG_YOLO_RUNTIME="direct"
export ZHIHANG_YOLO_PYTHON="/opt/yolo/bin/python"
```

不建议在 ROS 系统 Python 中无隔离地升级 NumPy 或 OpenCV，因为可能破坏 [cv_bridge](#)。已有 ROS/YOLO 依赖冲突时，优先 conda 或 venv。

7.5 模型类别检查

[verify_yolo_runtime.py](#) 默认要求权重包含：

```
prius_hybrid
prius_hybrid_camo
car_lexus
car_opel
fire_truck
person_white
suv_camo
person_red
```

权重缺少任何类别时，正式部署检查失败。模型类别名称必须与配置一致，不能仅靠 class id 顺序猜测。

7.6 三进程并发容量测试

```
bash benchmark_three_yolo_capacity.sh \
  --model "$HOME/yolo_models/best.pt" \
  --iterations 20 \
  --minimum-fps 10
```

输出目录含三份 JSON、三份日志和 [summary.json](#)。该测试能暴露显存不足和三进程争用，但它使用合成图像，不替代正式启动时的三路真实处理链屏障。

7.7 导出可复现环境

```
bash export_yolo_runtime_manifest.sh
```

保存：

- OS、内核和架构；
- Python、Torch、CUDA、cuDNN、Ultralytics、OpenCV 和 NumPy 版本；

- GPU 型号、驱动和显存；
- `pip freeze`；
- conda 环境 YAML；
- 模型 SHA-256。

这些信息应随总结报告归档，便于赛前电脑更换和答辩中的实时性说明。

8. 完整安装流程

8.1 解压和包内自检

```
cd ~/下载
tar -xzf zhihang_adaptive_search_v6_7_19_portable_complete.tar.gz
cd zhihang_adaptive_search_v6_7_19_portable_bundle

python3 validate_package.py
```

期望结尾：

```
ALL V6.7.19 CHECKS PASSED
```

8.2 生成 profile

```
bash configure_portable_machine.sh \
  --workspace "$HOME/xdroner_competition_ws" \
  --xdroner-root "$HOME/XTDrone" \
  --px4-root "$HOME/PX4_Firmware" \
  --model "$HOME/yolo_models/best.pt" \
  --runtime conda \
  --conda-env yolo26 \
  --record-bag \
  --install-user-profile
```

检查：

```
cat machine_profile.env
cat machine_profile_report.json
```

特别确认：

```
ZHIHANG_MESSAGE_WS_SETUP=$HOME/zhihang_ws/devel/setup.bash
ZHIHANG_PX4_LAUNCH_FILE=auto
ZHIHANG_YOLO_MODEL=/home/<用户名>/yolo_models/best.pt
ZHIHANG_YOLO_DEVICES=...
```

8.3 安装与编译

```
bash install_portable.sh
```

该脚本执行：

12. `validate_package.py`；
13. 备份已有 `zhihang_adaptive_search_v6`；
14. 记录受保护通信文件哈希；
15. 复制完整 ROS 包；
16. `catkin_make`；

17. 验证 ROS 包路径和配置；
18. 再次比较受保护文件哈希；
19. 加载真实模型做推理检查。

只安装代码、暂不验证 YOLO：

```
bash install_portable.sh --skip-yolo-check
```

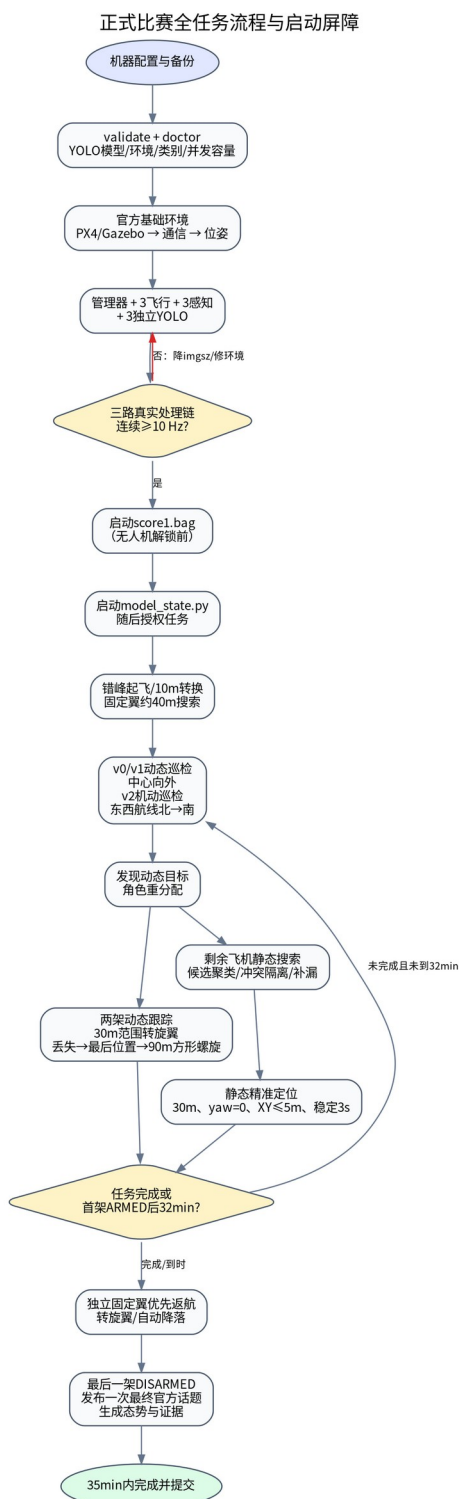
8.4 全面诊断

```
bash doctor_portable.sh
```

仿真已启动时增加实时话题检查：

```
bash doctor_portable.sh --live
```


9. 正式任务全流程



正式任务流程

9.1 启动命令

```

cd ~/下载/zhihang_adaptive_search_v6_7_19_portable_bundle

bash launch_portable_formal_one_click.sh \
  --scene scene_001_adaptive_v6_7_19_formal \
  --model "$HOME/yolo_models/best.pt" \

```

```
--runtime-env yolo26 \
--seed -1 \
--record-bag
```

不启动 QGC:

```
bash launch_portable_formal_one_click.sh \
--model "$HOME/yolo_models/best.pt" \
--runtime-env yolo26 \
--record-bag \
--no-qgc
```

9.2 自动启动顺序

20. 归一化场景、模型、环境和随机种子参数;
21. 检查模型、ROS 包、正式 YAML 和 YOLO 环境;
22. 打开 PX4/Gazebo;
23. 打开坐标/安全 guard;
24. 运行官方 XTDrone 通信;
25. 运行官方位姿脚本;
26. 可选启动 QGC;
27. 确认基础仿真、三机 MAVROS、位姿和相机话题可用;
28. 启动管理器和正式视觉动态状态估计器;
29. 启动三架飞机各自的 YOLO、飞行代理和感知代理;
30. 等待三路真实处理链连续达到 10Hz;
31. 启动 `score1.bag`;
32. 启动 `model_state.py`;
33. 确认目标运动话题发布;
34. 发布任务授权;
35. 三机错峰解锁、起飞并进入自主任务。

9.3 初始角色和航线

- `vtol0`: 动态巡检无人机, 从初始区中心向一侧外扩;
- `vtol1`: 动态巡检无人机, 从中心向另一侧外扩;
- `vtol2`: 机动巡检无人机, 按东西方向航线由北向南覆盖初始区, 与前两机的主方向正交。

固定翼搜索航段以直线巡航为主, 检测门控只在满足航向误差、横向偏差、速度和航段边界条件时接受搜索检测, 避免转弯段定位误差污染候选。

9.4 动态目标发现后的角色重分配

36. 第一动态目标被可靠定位: 机动巡检无人机 `vtol2` 离开当前航线, 成为动态跟踪无人机 1。
37. 第二动态目标被另一架空闲动态巡检无人机发现: 该机成为动态跟踪无人机 2。
38. 剩余一架动态巡检无人机转为静态搜索无人机。

39. `person_red` 类别可直接锁定；`suv_camo` 需经过五秒运动核验，防止误把 `prius_hybrid_camo` 静态车当作动态目标。

9.5 动态跟踪控制

- 固定翼以约 40m 高度快速接近；
- 只有水平距离进入 30m 范围才请求转旋翼；
- 转旋翼后先保持当前高度；
- 视觉目标确认后再平滑降至 30m 跟踪高度；
- 控制器使用位置滤波、目标速度前馈、相对速度阻尼、加速度限幅和命令滤波，减小姿态晃动；
- 只有目标足够近且信息新鲜时才根据目标调整 yaw；目标丢失后冻结 yaw 并继续飞向最后位置，不原地等待。

9.6 动态目标丢失和重捕获

40. 目标流过期后继续向最后已知/预测位置运动；
41. 到达前如重新检测，立即回到跟踪；
42. 未检测到时保持旋翼和当前约 40m 高度；
43. 按方形等距螺旋快速覆盖目标可能区域；
44. 航线间距 90m、速度 6m/s、机头沿轨迹切向；
45. 搜索半径随最后捕获时间和目标速度上界扩展；
46. 其他飞机上传同类高置信度且有定位的目标时，作为外部信息注入，更新中心并快速接近。

9.7 `suv_camo/prius_hybrid_camo` 混淆保护

- `suv_camo` 候选在 5 秒观察窗内无明显移动，则判为静态误检；
- 将其重新分类为 `prius_hybrid_camo` 精确候选；
- 释放临时动态跟踪角色；
- 根据是否已有另一架正式动态跟踪机，恢复一架或两架动态巡检角色；
- 各机从自己未完成航线的就近点继续；
- 新的 `prius_hybrid_camo` 信息若距离已确认 `suv_camo` 动态状态过近，拒绝更新，降低互相替代。

9.8 静态搜索和候选管理

- 静态搜索机首先保持有价值的当前航线；
- 使用高风险区、目标配对先验、已覆盖网格和未覆盖残余区生成后续路线；
- 相同类别、相近位置的报告合并，保留更高置信度和更稳定定位；
- 同类别但位置差异较大的报告保留为多个候选；
- 每类最多 5 个候选，按质量和距离排序逐一精准验证；
- 旋翼精准验证后无有效定位，候选标记为错检并保存原始图；

- 每个静态模型最终只保留一个最高置信度、已精准确认的位置。

9.9 静态精准定位

每个候选执行：

47. 固定翼接近；
48. 水平距离约 30m 时转旋翼；
49. 强制确认退出固定翼 HOLD 并恢复 OFFBOARD；
50. 到达冻结的搜索阶段目标位置上方 30m；
51. 控制 yaw 到 0°；
52. 水平误差≤5m、高度误差≤1m；
53. 条件稳定 3 秒；
54. 使用悬停期间检测结果更新最终位置；
55. `person_white` 悬停点在冻结位置基础上向西 10m，以减轻树遮挡；
56. 完成后在约 30m 高度转回固定翼继续任务或返航。

9.10 返航与结束

触发条件：

- 两个动态目标满足持续跟踪要求且六个静态目标均精准确认；或
- 首架飞机解锁后达到 32 分钟；或
- 管理器触发协调安全返航。

返航：

- 距离较远或刚完成旋翼任务时优先转固定翼；
- 飞向本机返航门点和穿越点；
- 转旋翼进入各自错开的降落点；
- 自动降落并上锁；
- 管理器继续监督直到全部飞机安全结束。

结束后：

57. 写入 `evaluation.json` 和 `final_results.json`；
58. 生成 `competition_final_results.json`，按“精确结果优先、可用结果保底”整理；
59. 官方结果桥接器一次发布静态/动态话题；一个分量为空不会阻断另一个分量；
60. 写入 `competition_topic_publish_evidence.json`；
61. 发布管理器任务完成消息；
62. 保留 bag 和全部证据文件供压缩提交。

10. 验证模式与正式模式

正式模式

- `selected_result_source: yolo_projected;`
- 运行 `vision_target_state_estimator.py`;
- 不启用真值 relay;
- 任务规划和控制只使用视觉定位及飞行器状态;
- 检测评估器关闭;
- 可按赛事要求将 Gazebo 状态录入 bag。

验证模式

- 可运行 `validation_target_state_relay.py`;
- 用真值对漏检、错检、正确检测和定位误差进行离线/旁路评估;
- 真值评估结果只写日志，不反馈规划和飞控;
- 不得将验证模式结果作为正式比赛输出。

验证启动用于开发，不应替代正式一键命令。

11. 运行状态与关键日志

11.1 正常启动标志

```
[ARG-NORMALIZED] model=/home/.../best.pt
[OK] PX4 ROS package: ...
[OK] vehicle 0 preflight passed
[READY] v0 ...
[READY] v1 ...
[READY] v2 ...
[WAIT] all nodes/endpoints exist; waiting for three real YOLO camera pipelines...
[OK] ... application ready
[OK] model_state.py running ...
[OK] ... mission start barrier published
```

11.2 动态跟踪关键事件

```
TRACK_FW_30M_TRANSITION_GATE_REACHED
TRANSITION_MC_COMPLETE
DYNAMIC_TARGET_MOTION_CONFIRMED
TRACK_TARGET_STREAM_STALE
TRACK_LOSS_CONTINUE_TO_LAST_KNOWN
TRACK_REACQUISITION_PASS_START
TRACK_TARGET_REACQUIRED
EXTERNAL_TARGET_INFORMATION_INJECTED
```

11.3 静态精准定位关键事件

```
STATIC_CANDIDATE_GROUP_CREATED
STATIC_CANDIDATE_NEARBY_MERGED
```

```

STATIC_VERIFY_HOLD_GUARD_TRIGGERED
OFFBOARD_HOLD_RECOVERY_ATTEMPT
STATIC_CANDIDATE_PRECISE_CONFIRMED
STATIC_CANDIDATE_REJECTED
STATIC_CONFIRMED_FINAL
STATIC_VERIFY_COMPLETE

```

11.4 官方输出标志

```
OFFICIAL_TOPICS_PUBLISHED first-and-final static=6 dynamic=4 type=<消息类型>
```

现场确认：

```

rostopic echo -n 1 /zhihang2026/static_targets/pose
rostopic echo -n 1 /zhihang2026/dynamic_targets/pose

```

由于首次消息用于评分，不要在正式比赛前向这两个话题手工发布测试消息。正式联调应使用隔离的 ROS master 或重新启动完整官方环境。

12. 输出目录与提交材料

默认管理器输出：

```
~/zhihang_search_runs_v6/<mission_id>/
```

默认单机输出：

```
~/zhihang_vehicle_runs_v6/
```

主要文件：

- `search_plan.json`：初始区域、航线和角色；
- `coverage_samples.jsonl`：覆盖轨迹采样；
- `yolo_reports.jsonl`：检测性能；
- `target_localization_reports.jsonl`：定位流；
- `static_candidate_state.json`：多候选状态；
- `static_candidate_events.jsonl`：合并、拒绝、确认；
- `flight_results.json`：三机飞行结果；
- `final_results.json`：全部最终任务数据；
- `evaluation.json`：任务时长、覆盖、跟踪和检测统计；
- `competition_final_results.json`：官方话题桥接输入；
- `competition_topic_publish_evidence.json`：官方发布证据；
- 原始图、标注图和静态拒绝原图；
- `score1.bag`：官方指定日志。

建议赛后：

```
bash analyze_latest_run.sh
```

并将任务目录、bag、态势图、录屏、环境清单和说明书统一归档。

13. 常见故障处理

13.1 模型路径显示为`model= ~/...`

原因：参数包含前导空格，或引号内使用了带空格的~。

正确：

```
--model "$HOME/yolo_models/best.pt"
```

V6.7.19 会清理前后空格，但仍应使用\$HOME 绝对展开方式。

13.2 `rospack: package 'zhihang\_adaptive\_search\_v6' not found`

```
cd ~/xtdrone_competition_ws
source /opt/ros/noetic/setup.bash
catkin_make
source devel/setup.bash
rospack profile
rospack find zhihang_adaptive_search_v6
```

随后检查 profile 的 ZHIHANG_WS。

13.3 找不到`adaptive\_search\_formal.yaml`

通常是子终端继承了旧环境标记但没有真正 source 工作空间。V6.7.19 每个子终端都会重新验证 ROS overlay。仍报错时：

```
bash source_zhihang_ros_env.sh
```

查看输出的包路径和配置路径。

13.4 找不到官方消息类型

检查：

```
source /opt/ros/noetic/setup.bash
source ~/zhihang_ws/devel/setup.bash
rosmmsg list | grep -i zhihang
```

profile 应有：

```
export ZHIHANG_MESSAGE_WS_SETUP="$HOME/zhihang_ws/devel/setup.bash"
```

如自动发现有多个候选，可在两个YAML的 `competition_output/outer_message_type` 显式填写实际包名/消息名。

13.5 CUDA 不可用

```
nvidia-smi
conda run -n yolo26 python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available())"
```

驱动可见但 Torch 返回 False，通常是安装了 CPU 版 Torch 或 CUDA wheel 不匹配。不要通过修改系统 CUDA 路径盲目修复；重新安装与驱动兼容的官方 PyTorch wheel。

13.6 CUDA OOM

```
nvidia-smi
bash safe_cleanup_previous_run.sh
```

确认无旧 YOLO 进程，再将尺寸改为：

```
export ZHIHANG_YOLO_ADAPTIVE_SIZES="512,416"
```

或合理分配多 GPU。

13.7 三路处理链不足 10Hz

依次检查：

63. 相机原始话题是否 $\geq 10\text{Hz}$ ；
64. 单 YOLO 真实模型 FPS；
65. 三进程并发 FPS；
66. JPEG 编码、TCP 和 ROS 定位处理耗时；
67. GPU 显存/功耗/温度；
68. 是否有屏幕录制或其他 GPU 程序抢占；
69. 自适应尺寸是否生效。

正式启动器不会绕过 10Hz 屏障。

13.8 QGC 显示 `Invalid offboard setpoint`

V6.7.19 保留两类保护：

- 固定翼模式禁止发送无效零速度交接 setpoint；
- HOLD 出现时重新预发送有效 setpoint 并请求 OFFBOARD 恢复。

若仍出现，提取对应飞机日志中以下事件前后至少 30 秒：

```
FW_ZERO_VELOCITY_HANOVER_BLOCKED
OFFBOARD_HOLD_RECOVERY_ATTEMPT
STATIC_VERIFY_HOLD_GUARD_TRIGGERED
```

同时查看：

```
rostopic echo /standard_vtol_<ID>/mavros/state
rostopic echo /standard_vtol_<ID>/mavros/local_position/pose
```

13.9 动态跟踪反复调 yaw 后丢失

确认配置仍使用：

- 近距离/新鲜目标才允许 yaw 跟随；
- 目标丢失后冻结 yaw；
- 不原地悬停，继续向最后位置运动；
- 重捕获机头沿轨迹方向。

不要把 yaw 增益单独调大来追求框居中，否则会重新引入姿态晃动和视场丢失。

13.10 静态精准定位陷入固定翼 HOLD

查看是否发生：


```

STATIC_VERIFY_HOLD_GUARD_TRIGGERED
OFFBOARD_HOLD_RECOVERY_ATTEMPT
TRANSITION_MC_COMPLETE

```

若无恢复事件，检查 MAVROS 模式服务和 PX4 转换状态；若有恢复但重复进入 HOLD，检查是否有其他节点向同一飞机发布控制指令。

14. 赛前完整检查表

T-1 天

- ☐ 备份可运行版本和 ROS 包；
- ☐ 校验压缩包 SHA-256；
- ☐ 固定 GPU 驱动、Torch、Ultralytics 和模型版本；
- ☐ 导出运行时 manifest；
- ☐ 运行 `validate_package.py`；
- ☐ 运行 `doctor_portable.sh`；
- ☐ 运行三 YOLO 并发容量测试；
- ☐ 完成一次 35 分钟完整任务；
- ☐ 检查六静态、两动态、官方话题和 bag；
- ☐ 检查磁盘空间和录屏空间。

比赛启动前

- ☐ 无旧 PX4、Gazebo、MAVROS、管理器、飞行代理、YOLO worker；
- ☐ 官方通信文件哈希与基准一致；
- ☐ `score1.bag` 旧文件已备份；
- ☐ 模型路径无空格错误；
- ☐ `zhihang_ws` 已 source 并能列出消息；
- ☐ 三个 YOLO 端口未被占用；
- ☐ QGC 三机连接正常；
- ☐ 相机话题均 $\geq 10\text{Hz}$ ；
- ☐ 不向官方结果话题手工发布测试消息。

任务结束后

- ☐ 三机均 `disarmed`；
- ☐ 官方两话题各有一条最终消息；

- ☐ `competition_topic_publish_evidence.json` 存在；
- ☐ `score1.bag` 存在并包含官方 9 话题；
- ☐ 压缩 bag；
- ☐ 生成态势图；
- ☐ 归档录屏、总结报告、代码、环境 manifest 和运行目录。

15. 回滚

自动安装器会输出备份目录。手工回滚示例：

```
cd ~/xtdrone_competition_ws/src
rm -rf zhihang_adaptive_search_v6
cp -a ~/xtdrone_backups/zhihang_adaptive_search_v6_before_v6_7_19_<时间戳> \
  zhihang_adaptive_search_v6

cd ~/xtdrone_competition_ws
source /opt/ros/noetic/setup.bash
catkin_make
```

profile 不会修改 ROS 源码，可直接删除：

```
rm -f ~/下载/zhihang_adaptive_search_v6_7_19_portable_bundle/machine_profile.env
rm -rf ~/.config/zhihang_v6_7_19
```

不要通过回滚算法包覆盖赛事官方通信脚本。

16. 验证状态与限制

交付包已完成：

- Python 语法、Shell 语法、ROS launch XML 和 YAML 解析；
- V6.7.18 核心控制行为回归测试；
- 30m 动态转换、90m/6m/s 方形螺旋、动态角色回滚；
- 静态候选聚类、原图证据、prios/suv 隔离；
- HOLD/无效 setpoint 防护；
- 跨电脑 profile、终端后端和 YOLO 运行时测试；
- 赛事 9 话题 bag 和官方输出桥接代码检查；
- 正式模式目标真值防火墙检查。

容器中没有用户的 PX4/Gazebo、三路相机、`zhihang_ws` 实际消息、GPU 和真实 `best.pt`，因此以下内容必须在目标电脑验证：

- catkin 实际编译；
- 官方自定义消息自动发现；
- 模型真实类别；
- 三进程显存和真实端到端 FPS；

- VTOL 转换、Offboard、跟踪、精准悬停和降落；
- 官方评分程序接收结果。

正式启动器通过运行屏障阻止三路真实处理链不达标时开始任务，但它不能替代完整赛场联调。

17. 完整代码索引

全部源代码、配置、测试和启动脚本均在本交付包内，详见 [SOURCE_CODE_INDEX.md](#) 和

[SHA256SUMS.txt](#)。核心目录：

```
zhihang_adaptive_search_v6/  
  config/  
  launch/  
  scripts/  
  src/zhihang_adaptive_search_v6/  
terminal_commands/  
tests/  
docs/images/
```

常用入口：

```
configure_portable_machine.sh  
setup_yolo_runtime.sh  
install_portable.sh  
doctor_portable.sh  
benchmark_three_yolo_capacity.sh  
launch_portable_formal_one_click.sh  
status_one_click.sh  
abort_all.sh  
analyze_latest_run.sh
```

—— V6.7.19 Portable 说明书结束 ——